

CPU Simulation

Programmentwurf

Studiengang Informatik

Duale Hochschule Baden-Württemberg Karlsruhe

Marc Schillinger

Eingereicht am: 15.06.2025

Matrikelnummer, Kurs: 3449698, TINF22B2

Unternehmen: Atruvia AG, 76131, Karlsruhe

Betreuer an der DHBW: Lars Briem

Inhalt

1 Kapitel 1: Einführung	1
1.1 Übersicht über die Applikation	1
1.2 Wie startet man die Applikation?	2
1.3 Wie testet man die Applikation?	2
2 Kapitel 2: Clean Architecture	4
2.1 Was ist Clean Architecture?	4
2.2 Analyse der Dependency Rule	5
2.2.1 Positiv-Beispiel	5
2.2.2 Negativ-Beispiel	5
2.3 Analyse der Schichten	6
2.3.1 Schicht: Interface-Schicht	6
2.3.2 Schicht: Application Code	7
3 Kapitel 3: SOLID	8
3.1 Analyse Single-Responsibility-Principle (SRP)	8
3.1.1 Positiv-Beispiel	8
3.1.2 Negativ-Beispiel	9
3.2 Analyse Open-Closed-Principle (OCP)	9
3.2.1 Positiv-Beispiel	9
3.2.2 Negativ-Beispiel	10
3.3 Analyse LSP, ISP oder DIP	10
3.3.1 Positiv-Beispiel	10
3.3.2 Positiv-Beispiel	11
4 Kapitel 4: Weitere Prinzipien	12
4.1 Analyse GRASP: Geringe Kopplung	12
4.1.1 Positiv-Beispiel	12
4.1.2 Negativ-Beispiel	13
4.2 Analyse GRASP: Hohe Kohäsion	14
4.3 Don't Repeat Yourself (DRY)	14
5 Kapitel 5: Unit Tests	16
5.1 10 Unit Tests	16
5.2 ATRIP: Automatic	17
5.3 ATRIP: Thorough	18
5.3.1 Positiv-Beispiel	18

5.3.2	Negativ-Beispiel	19
5.4	ATRIP: Professional	20
5.4.1	Positiv-Beispiel	20
5.4.2	Negativ-Beispiel	20
5.5	Code Coverage	21
5.6	Fakes und Mocks	22
5.6.1	Beispiel 1	22
5.6.2	Beispiel 2	23
6	Kapitel 6: Domain Driven Design	24
6.1	Ubiquitous Language	24
6.2	Entities	26
6.3	Value Objects	27
6.4	Repositories	27
6.5	Aggregates	27
7	Kapitel 7: Refactoring	29
7.1	Code Smells	29
7.1.1	Code Smell 1	29
7.1.2	Code Smell 2	29
7.2	2 Refactorings	30
7.2.1	Refactoring 1	30
7.2.2	Refactoring 2	31
8	Kapitel 8: Entwurfsmuster	32
8.1	Entwurfsmuster: Builder	32
8.2	Entwurfsmuster: Strategie	33
A	Literatur	34
B	Abkürzungen	35
C	Glossar	36



Kapitel 1: Einführung

1.1 Übersicht über die Applikation

Entwicklungsumgebung	IntelliJ IDEA 2024.1.2 (Community Edition)
Build-Management-Tool	Gradle 8.12
Repository	ase-cpu-sim

Tabelle 1 – Eckdaten des Projekts

Die Applikation ist eine Simulation einer CPU. Betrachtet wurden wesentliche Komponenten, welche in der Betriebssystemevorlesung aus Semester 4 besprochen wurden. Zur Vereinfachung wurde ein abgesteckter Rahmen gewählt und Funktionen in der Tiefe nicht vergleichbar wie bei echten CPUs umgesetzt.

Anhand des Codes und der Bedienung der Applikation kann ein Basisverständnis von abläufen innerhalb einer CPU erlangt werden. Funktionen wie das Lesen und Schreiben von einzelnen Byte, sowie einfache Rechenoperationen, wie dem Addieren und Subtrahieren wurden umgesetzt. Die Applikation wurde so konzipiert das Erweiterungen leicht hinzugefügt werden können. Weitere Funktionen, wie das Multiplizieren oder das logische Und sind somit umsetzbar.

Zentral ist die ControlUnit für alle Abläufe innerhalb der simulierten CPU zuständig. Sie nutzt den Decoder, um Commands (Nutzereingaben) zu decoden in Instructions, welche dann von den einzelnen Komponenten der CPU ausgeführt werden können. Zu diesen Komponenten gehören in dieser Implementierung die ALU und ein Register.

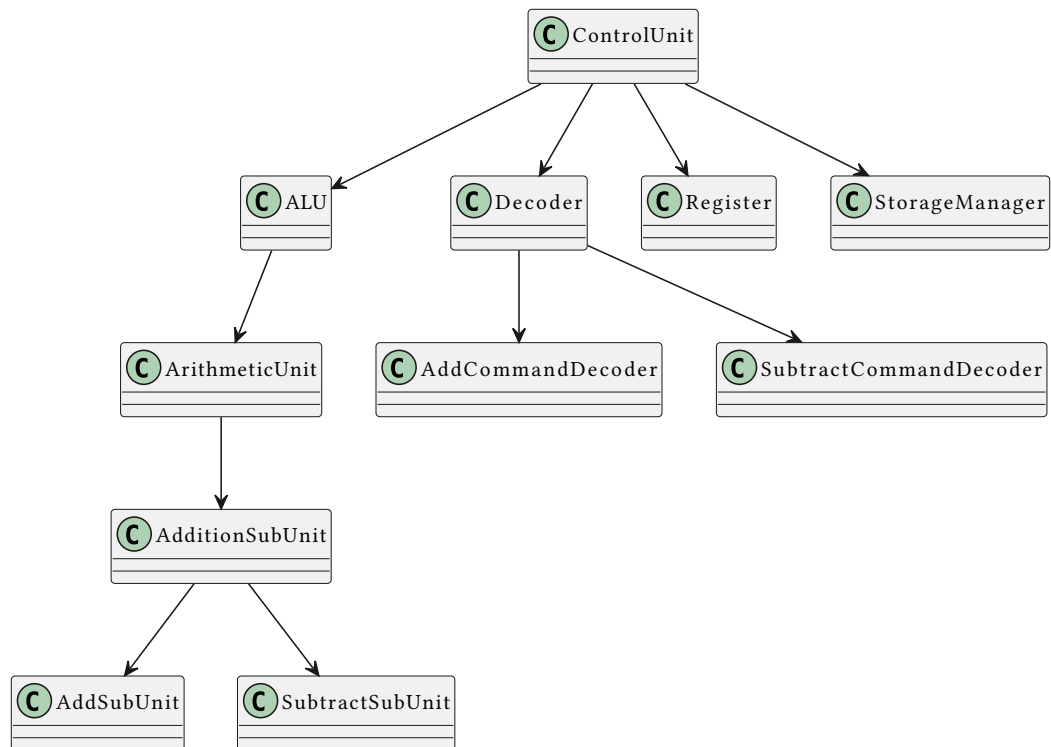


Abbildung 1 – CPUSimulation grobe Übersicht

1.2 Wie startet man die Applikation?

1. Starte einen Terminal
2. Navigiere in das Root-Verzeichnis des Projekts
3. Führe den Command aus Listing 1 aus

```
1 ./gradlew run --console=plain
```

Listing 1 – Applikation Start-Command

1.3 Wie testet man die Applikation?

1. Starte einen Terminal
2. Navigiere in das Root-Verzeichnis des Projekts
3. Führe den Command aus Listing 2 aus

```
1 ./gradlew test
```

Listing 2 – Applikation Test-Command



Kapitel 2: Clean Architecture

2.1 Was ist Clean Architecture?

Clean Architecture strebt an möglichst langlebigen Code zu strukturieren und zu schreiben. Langlebiger Code wird durch Flexibilität, Skalierbarkeit und guter Wartung definiert. Hierfür wird ein mehrschichtiges System angestrebt, welches klare Schnittstellen und Abhängigkeiten vorsieht. Gedacht wird dieses System wie eine Zwiebel. Im Kern befindet sich die Anwendungslogik und in der äußeren Schichten die Peripheriegeräte. Abhängigkeiten gelten immer nur von einer äußeren zu einer weiter innen liegenden Schicht.

2.2 Analyse der Dependency Rule

2.2.1 Positiv-Beispiel

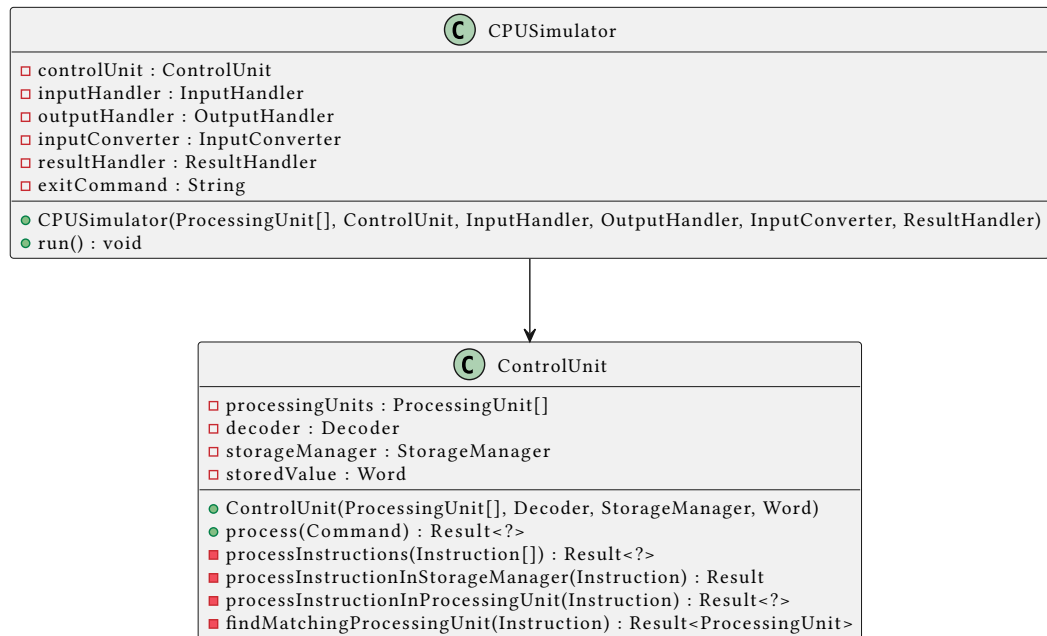


Abbildung 2 – DependencyRule Positivbeispiel UML

Der CPUSimulator-Klasse liegt innerhalb des Schichtenmodells weiter außen als die ControlUnit-Klasse. Somit darf nach der Dependency Rule die ControlUnit-Klasse nichts von der CPUSimulator-Klasse wissen. Anhand von Abbildung 2 kann man sehen das diesem Prinzip gefolgt wird.

2.2.2 Negativ-Beispiel

Es wurde kein Negativ-Beispiel innerhalb des Codes gefunden. Somit wird an dieser Stelle ein weiteres Positiv-Beispiel eingefügt.

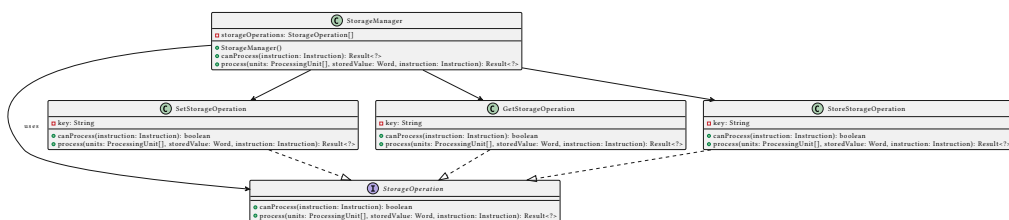


Abbildung 3 – StorageManager Positivbeispiel UML

Der StorageManager ist ebenfalls ein positives Beispiel für die Dependency Rule. Wie in Abbildung 3 zu sehen kennen die StorageOperation-Klassen die StorageManager-Klasse nicht. Da in der Schichtendarstellung, die StorageOperation-Klassen tiefer liegen, ist die Dependency Rule erfüllt.

2.3 Analyse der Schichten

2.3.1 Schicht: Interface-Schicht

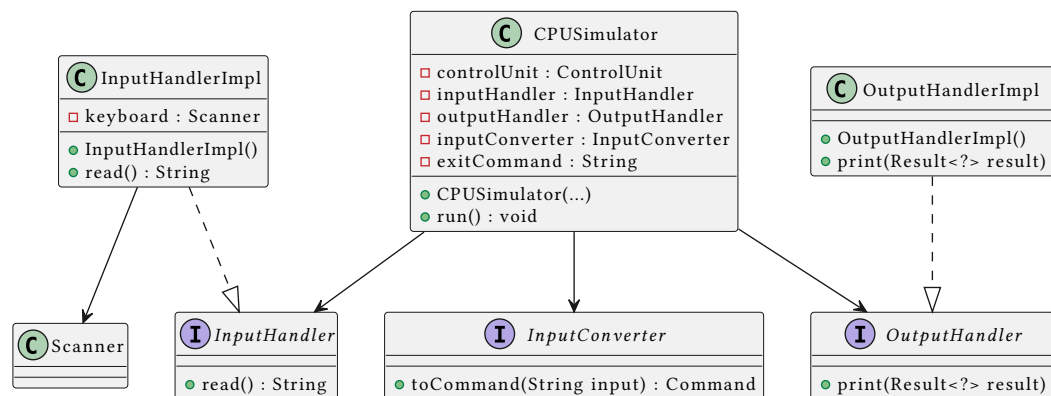


Abbildung 4 — Adapters-Schicht UML

Die Adapter-Schicht liegt zwischen der Plugin-Schicht, welche Geräte, Drittsysteme oder GUIs umfasst, und dem Application Code. Sie vermittelt zwischen den Plugins und den Use Cases.

In diesem Fall befinden sich die gezeigten Klassen in der Adapter-Schicht (siehe Abbildung 4). `InputHandler` und `OutputHandler` sind dafür verantwortlich mit den Plugins zu kommunizieren. Die `InputHandlerImpl`-Klasse greift hierbei auf die Tastatureingaben zu. Die `OutputHandlerImpl`-Klasse sorgt dafür dass die Ausgaben des Systems an die Console weitergeleitet werden.

Die Klasse `CPUSimulation` kümmert sich um die Verbindung zu den Use Cases und koordiniert die Laufreihenfolge der einzelnen Komponenten. Ergo wann auf eine Eingabe gewartet wird, was mit dieser geschieht und was wiederum dem Nutzer zurückgemeldet wird.

2.3.2 Schicht: Application Code

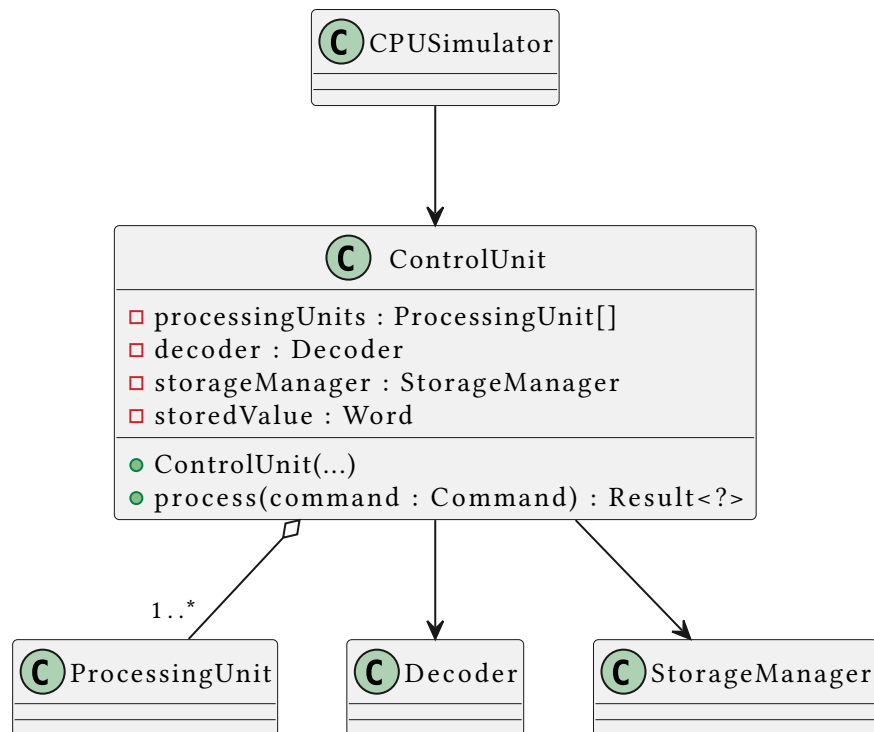


Abbildung 5 – Application Code UML

Die Application-Code-Schicht liegt zwischen der Adapter-Schicht und dem Domain Code. Sie beschreibt die Anwendungsregeln, unabhängig von Ein- oder Ausgabegeräten, Datenbanken oder Frameworks.

In dieser Schicht befinden sich die Use Cases des Systems (siehe Abbildung 5). Sie definieren, was das System tun soll, nicht wie es technisch umgesetzt wird. Die Klassen in dieser Schicht orchestrieren den Ablauf von Prozessen, rufen Entities auf und koordinieren deren Zusammenarbeit.

In diesem Fall übernimmt die Klasse **ControlUnit** die Rolle eines zentralen Use Cases. Sie verarbeitet eingehende Befehle, lässt diese über einen Decoder in ausführbare Anweisungen übersetzen und verteilt diese an die zuständigen Einheiten (z.B. ALU oder Register).

3

Kapitel 3: SOLID

3.1 Analyse Single-Responsibility-Principle (SRP)

3.1.1 Positiv-Beispiel

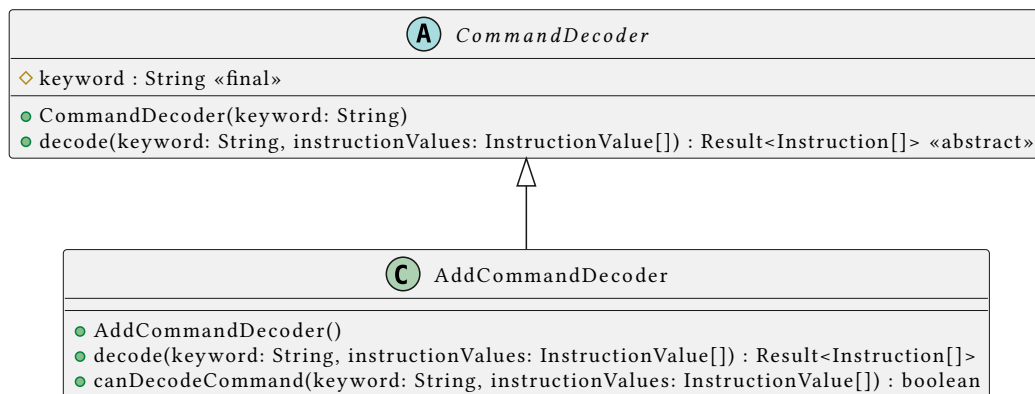


Abbildung 6 — AddCommandDecoder-Klasse UML

Die AddCommandDecoder-Klasse ist allein dafür zuständig einen Command, bestehend aus einem Keyword und mehreren Parametern (InstructionValues), in eine Instruction umzuwandeln.

Jeder Command der decodiert werden kann wird von einer eigenständigen Klasse decodiert.

3.1.2 Negativ-Beispiel

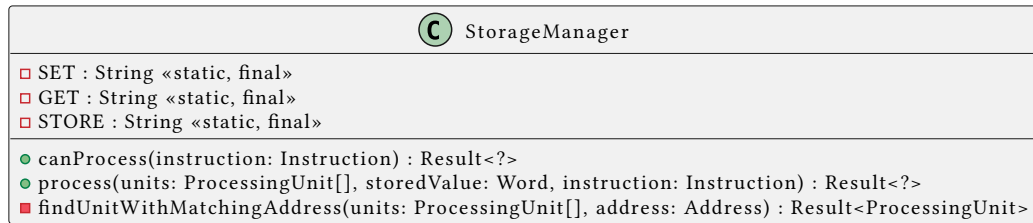


Abbildung 7 – StorageManager-Klasse UML

Der StorageManager (siehe Abbildung 7) ist Teil der ControlUnit und ist dafür zuständig Daten zu Setzen, Holen und Zwischenspeichern. Er übernimmt die Ausführung aller drei Funktionen selbst und verwendet nicht weitere Unterklassen, um die jeweilige Umsetzung von sich zu trennen.

3.2 Analyse Open-Closed-Principle (OCP)

3.2.1 Positiv-Beispiel

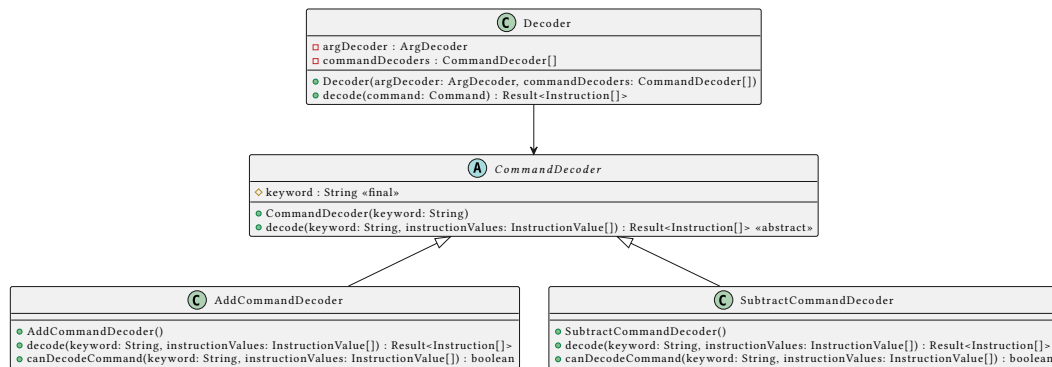


Abbildung 8 – Decoder-Klasse UML

Die Decoder-Klasse (siehe Abbildung 8) zeigt hierfür ein gutes Beispiel. Um offen für Erweiterungen und geschlossen für Modifizierungen zu sein nutzt der Decoder die Klasse **CommandDecoder** (siehe auch Abbildung 6). Diese ermöglicht es dass unterschiedliche **CommandDecoder** eingesetzt werden können. Die Funktion eines Decoders bleibt immer gleich, aber es bleiben verschiedene Implementierungen eines **CommandDecoder**s somit möglich.

3.2.2 Negativ-Beispiel

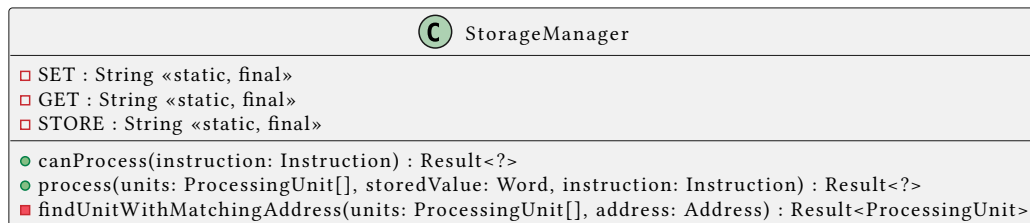


Abbildung 9 – StorageManager-Klasse UML

Der StorageManager ist ein negatives Beispiel für OCP. Er kümmert sich um mehrere Instruction's, das sind:

- SET: Setzt einen Speicherplatz (StorageSpace) auf einen Wert
- GET: Holt den Wert eines Speicherplatzes (StorageSpace)
- STORE: Speichert einen Wert aus einer Nutzereingabe zwischen in der ControlUnit

Um OCP zu erfüllen müssten diese einzelnen Funktionalitäten voneinander getrennt werden und nicht wie in diesem Beispiel alle in einer Klasse behandelt werden. Somit könnte deren Funktionalität auch unabhängiger getestet werden.

3.3 Analyse LSP, ISP oder DIP

3.3.1 Positiv-Beispiel

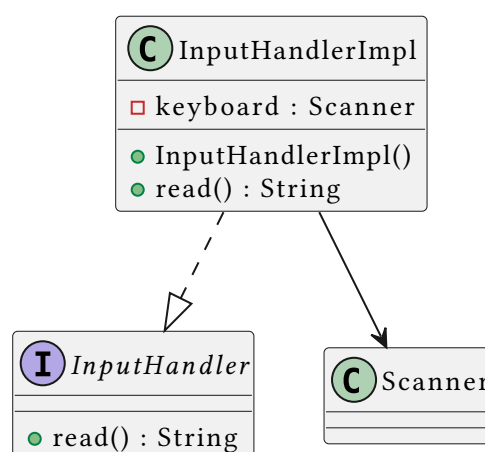


Abbildung 10 – InputHandler-Klasse UML

Für Liskov Substitution Principle ist der InputHandler ein gutes Beispiel (siehe Abbildung 10). Er wird genutzt um in der Auswahl der Eingabegeräte flexibel zu bleiben. Erwartet wird

ein String als Rückgabe. Aber wie genau dieser String aufgenommen wird ist hierbei nicht relevant. Da die aktuelle Implementierung in der Console stattfindet wurde die Klasse InputHandlerImpl umgesetzt, welche diese Funktionalität umsetzt und die Eingabedaten schließlich mit der Scanner-Klasse einholt. Falls sich das Eingabegerät ändern sollte kann problemlos eine weitere Implementierung für diesen Fall umgesetzt werden.

3.3.2 Positiv-Beispiel

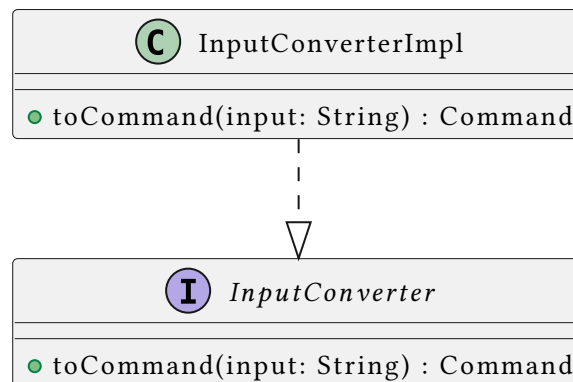


Abbildung 11 – InputConverter-Klasse

Die InputConverter-Klasse (siehe Abbildung 11) nutzt ebenfalls ein Interface, um flexibel in seiner Implementierung zu bleiben. Sobald sich eine neue Anforderung an die Erstellung von Commands aus dem Texteingaben des Nutzers ergibt, kann eine andere Implementierung umgesetzt werden. Somit erfüllt die InputConverter-Klasse ebenfalls das Liskov Substitution Principle.

4

Kapitel 4: Weitere Prinzipien

4.1 Analyse GRASP: Geringe Kopplung

4.1.1 Positiv-Beispiel

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4         ...  
5         Decoder decoder = new Decoder(new ArgDecoder(), new CommandDecoder[]{  
6             new AddCommandDecoder(),  
7             new SubtractCommandDecoder(),  
8             new SetCommandDecoder(),  
9             new GetCommandDecoder(),  
10            new StoreCommandDecoder(),  
11        });  
12        ...  
}
```

Listing 3 – Erstellung Decoder in main-Methode

Ein positives Beispiel für eine geringe Kopplung nach GRASP ist die Decoder-Klasse (siehe Abbildung 8). Der Decoder bekommt alle seine CommandDecoder, sowie den ArgDecoder bei Erstellung übergeben und muss sich nicht selbst darum kümmern. Somit können bspw.

bei Tests gezieht Funktionen getestet werden ohne alle Funktionalitäten aus der vollständig instanziierten Laufzeitumgebung zu übernehmen.

4.1.2 Negativ-Beispiel

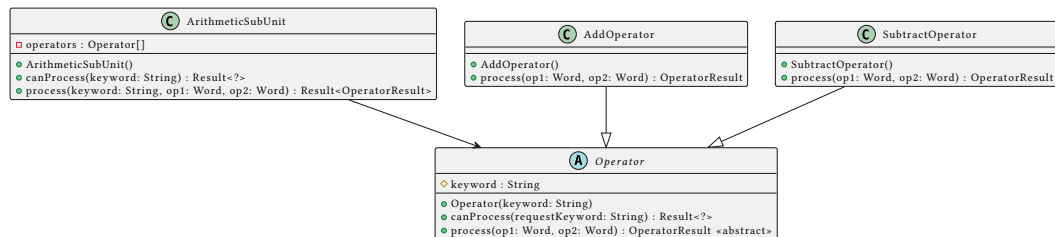


Abbildung 12 – ArithmeticProcessingUnit-Klasse

```

1 public class ArithmeticSubUnit implements SubUnit {
2
3     private Operator[] operators;
4
5     public ArithmeticSubUnit() {
6         operators = new Operator[]{new AddOperator(), new
7         SubtractOperator()};
8     }
9     ...
  
```

Listing 4 – ArithmeticSubUnit Constructor

GRASP findet sich in der ArithmeticProcessingUnit-Klasse nicht wieder (siehe Abbildung 12). Innerhalb des Constructors der ArithmeticSubUnit (siehe Listing 4) sieht man das die verwendeten Operatoren der ArithmeticSubUnit nicht über die Contstructor-Attribute empfangen werden, sondern bereits festgelegt sind. Somit ist die Zusammensetzung der ArithmeticProcessingUnit-Klasse nicht flexibel. Diese Implementierung verletzt GRASP wurde aber bewusst so umgesetzt. Diese Klasse ist per Definition immer an die selben Klassen gebunden und somit wurde sich entschieden das diese direkt im eigenen Code eingebettet werden.

Es wäre auch denkbar, wie bei dem positiven Beispiel, die Klassen als Attribute über den Constructor zu empfangen. Somit wäre GRASP nicht mehr verletzt.

4.2 Analyse GRASP: Hohe Kohäsion

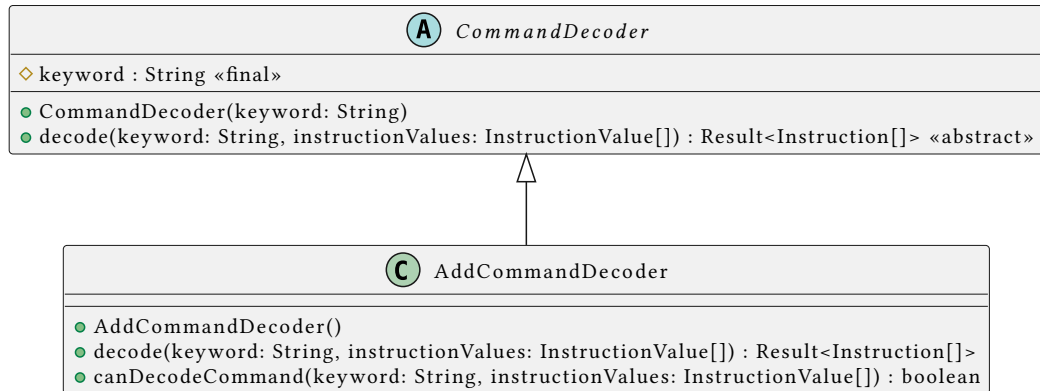


Abbildung 13 — AddCommandDecoder-Klasse

Die AddCommandDecoder-Klasse ist ein einfaches Beispiel für hohe Kohäsion (siehe Abbildung 13). Sie umfasst lediglich eine Methode neben dem Constructor und ist somit maximal auf die Umsetzung einer Aufgabe fokussiert.

4.3 Don't Repeat Yourself (DRY)

```

1  public Result<?> canProcess(Instruction instruction) {
2      ...
3      if(instruction.getKeyword().contentEquals(SET)) {
4          InstructionValue[] instructionValues = instruction.getValues();
5          ...
6      } else if(instruction.getKeyword().contentEquals(GET)) {
7          ...
8      } else if(instruction.getKeyword().contentEquals(STORE)) {
9          ...
10     } else {
11         ...
12     }
13 }
  
```

Listing 5 — StorageManager zuvor

```
1 public Result<?> canProcess(Instruction instruction) {  
2     for(StorageOperation storageOperation : storageOperations) {  
3         if(storageOperation.canProcess(instruction)) {  
4             return Result.ok();  
5         }  
6     }  
7 }
```

Listing 6 — StorageManager danach

[Link zum Commit](#)

Listing 5 zeigt wie die StorageManager-Klasse vor der Änderung aussah. Es wurde für jeden Case den es gab eine If-Klausel eingeführt. Das führte zu einer Codewiederholung. Durch die Anpassung die einzelnen Fälle in jeweils einzelne Klassen auszulagern, konnte die Komplexität der Klasse verringert werden und DRY eingehalten werden (siehe Listing 6).

5

Kapitel 5: Unit Tests

5.1 10 Unit Tests

Unit Test	Beschreibung
StorageSpaceTest = testConstructorWithWord()	Testet ob die StorageSpace-Klasse die im Constructor übergebenen Werte übernimmt.
AddOperatorTest = testeEinfacheAddition()	Testet ob AddOperator eine einfache Addition (ohne Übertrag) zweier Words ausführen kann.
AddOperatorTest = testeAdditionMitÜberlauf()	Testet ob AddOperator eine Addition mit Übertrag zweier Words ausführen kann.
ALUTest = testCanProcess()	Testet ob die ALU ein valides Keyword verarbeiten kann und ein invalides Keyword nicht verarbeiten kann.
ALUTest = testProcessAddition()	Testet ob eine Addition-Instruction erfolgreich verarbeitet werden kann. Die ALU reicht diese Anfrage an den AddOperator weiter. Diese Weiterleitung wird getestet.
AddCommandDecoderTest = decode_shouldReturnInstructions _whenValidArgs()	Testet ob der AddCommand in die richtige Kombination aus Instructions übersetzt wird.
GetCommandDecoderTest = testMissingAddress()	Testet ob ein Fehler auftritt beim decoden, wenn die Adresse nicht spezifiziert wurde.

Unit Test	Beschreibung
ArgsDecoderTest = testDecodeWordAndAddress()	Testet ob die übergebenen Argumente (Word und Address) richtig erkannt und übertragen werden in ihre Klassenrepräsentation.
CommandTest = testeNurMitKeyword()	Testet ob ein Command nur von einem Keyword erstellen kann.
CommandTest = testeCommandErstellungMit CommandBuilder()	Testet ob man einen Command über den CommandBuilder erstellen kann.

5.2 ATRIP: Automatic

Die Tests lassen sich automatisiert ausführen durch den Befehl im Hauptverzeichnis.

```
1 ./gradlew test
```

Listing 7 — Command zum ausführen der Tests

5.3 ATRIP: Thorough

5.3.1 Positiv-Beispiel

```
1  @Test
2  void decode_shouldReturnError_whenInvalidArgs() {
3      AddCommandDecoder decoder = new AddCommandDecoder();
4
5      InstructionValue[] values = new InstructionValue[] {
6          new InstructionValue(null, null),
7          new InstructionValue(null, null)
8      };
9
10     Result<Instruction[]> result = decoder.decode("add", values);
11
12     assertTrue(result instanceof Result.Error);
13     assertEquals("Invalid decoding of arg!",
14         ((Result.Error<Instruction[]>) result).getException().getMessage());
15 }
```

Um ein Beispiel für umfassendes Testen zu zeigen wurde die AddCommandDecoderTest-Klasse ausgewählt (siehe GitHub). In diesem Testbeispiel wird spezifisch getestet, wie die Funktion `decode` mit der Übergabe fehlender Argumente umgeht. Wie in Zeile 13 (siehe Listing 8) erkannt werden kann sieht man, dass die Fehlermeldung passend zur aufgetretenen Fehler ist. Innerhalb der Testklasse werden ebenso 3 weitere Testfälle behandelt, welche ein umfassendes Spektrum aus Funktionalitätsprüfung und Fehlerbehandlung beinhaltet.

5.3.2 Negativ-Beispiel

```
1  @Override
2  public Result<Word> read(Address address) {
3      Result<StorageSpace> findStorageSpaceResult =
4      findStorageSpace(address);
5      if(findStorageSpaceResult instanceof Result.Ok<StorageSpace>
6      findStorageSpaceOk) {
7          Word word = findStorageSpaceOk.getValue().getWord();
8          return Result.ok(word);
9      } else if(findStorageSpaceResult instanceof
10 Result.Error<StorageSpace> findStorageSpaceError){
11         return Result.error(findStorageSpaceError.getException());
12     } else {
13         return Result.error(new
14         UnexpectedResultException(this.getClass().getSimpleName()));
15     }
16 }
```

Ein schlechtes Beispiel für Thorough ist die Klasse ALU. Der oben gezeigte Code in Listing 9 ist nicht gut testbar, bzw wird nicht von der ALUTest-Klasse ausreichend getestet. Dies liegt am else-Branch. Dieser wird nur tätig sobald ein Objekt von der findStorageSpace-Methode zurückgeliefert wird, welche nicht den vorher definierten Cases entspricht. Da die Methode innerhalb der Klasse liegt ist es schwierig deren Funktion für einen Testfall zu ändern.

Somit ist dies ein schlechter Fall der umfangreichen Testung.

5.4 ATRIP: Professional

5.4.1 Positiv-Beispiel

```
1  @Test
2  void decode_shouldReturnOk_whenMatchingDecoderExists() {
3      ...
4  }
5
6  @Test
7  void decode_shouldReturnError_whenNoDecoderMatches() {
8      ...
9  }
```

Listing 10 — DecoderTest Test Cases

Als Positivbeispiel für Professional wurde die `DecoderTest`-Klasse ausgewählt (siehe GitHub). Sie verwendet einfach verständliche Methodennamen. Somit kann auf einen Blick verstanden werden was der Test macht.

5.4.2 Negativ-Beispiel

```
1  @Test
2  void testeCanProcessAdd() {
3      ArithmeticSubUnit subUnit = new ArithmeticSubUnit();
4      Result<?> result = subUnit.canProcess("add");
5      assertTrue(result instanceof Result.Ok<?>);
6  }
7
8  @Test
9  void testeCanProcessSubtract() {
10     ArithmeticSubUnit subUnit = new ArithmeticSubUnit();
11     Result<?> result = subUnit.canProcess("sub");
12     assertTrue(result instanceof Result.Ok<?>);
13 }
```

Listing 11 — ArithmeticSubUnitTest Test Cases

Die Testklasse `ArithmeticSubUnitTest` enthält keine Kommentare und die Methodennamen sind leicht irreführend. In beiden Testfällen wird suggeriert, dass getestet wird ob die

ArithmeticSubUnit-Klasse die Addition und Subtraktion selbst ausführen könne. Dies stimmt aber nicht. Die Klasse nutzt sogenannte Operator-Klassen, welche sich um die Umsetzung kümmern. Somit wird das Prinzip von Professional verletzt.

5.5 Code Coverage

Element	Classes	Methods	Lines	Branches
de.dbhw	80%	89%	78%	70%
io	100%	100%	94%	70%
units	100%	100%	93%	77%
utils.data	100%	100%	100%	100%

Ermittelt wurde die Code Coverage durch Gradle test und die IntelliJ-Funktion Run with Coverage.

Auf das gesamte Projekt de.dbhw betrachtet ist die Code Coverage nicht besonders hoch mit 80% aller Klassen und 70% der Branches.

Hierfür gibt es folgende Begründungen:

- Oft werden Results zurückgegeben, welche durch einen Instanzenvergleich geprüft werden. Hierbei kann es technisch betrachtet mehr als die implementierten Fälle Ok und Err kommen. Somit wurden manche Branches für die syntaktische Vollständigkeit der jeweiligen Klasse erstellt, obwohl sie nicht auftreten können im normalen Betrieb.
- Entity-Klassen wurden nicht getestet, da set- und get-Methodentests nicht relevant sind.

5.6 Fakes und Mocks

5.6.1 Beispiel 1

```
1  @Test
2  public void testeProcessSetErfolgreich() {
3      Address a1 = new Address("r", 1);
4      Word w = new Word();
5      InstructionValue v1 = new InstructionValue(a1, null);
6      Instruction instr = new Instruction("set", new InstructionValue[]
7      {v1});
8
9      ProcessingUnit mockUnit = mock(ProcessingUnit.class);
10     when(mockUnit.read(a1)).thenReturn(Result.ok(new Word()));
11     when(mockUnit.write(a1, w)).thenReturn(Result.ok());
12
13     StorageManager sm = new StorageManager();
14     Result<?> res = sm.process(new ProcessingUnit[]{mockUnit}, w, instr);
15
16     assertTrue(res instanceof Result.Ok<?>);
17 }
```

Listing 12 – StorageManagerTest-Klasse Mockbeispiel

StorageManagerTest verwendet in Test `testeProcessSetErfolgreich()` einen Mock für die verwendete `ProcessingUnit`. Dieser Test prüft ob der `StorageManager` die richtige Speicheradresse findet zu einer Speicherinstruction (SetInstruction genannt). Da keine spezifische `ProcessingUnit` hier mitgetestet werden soll wird ein Mock verwendet mit entsprechender `when(...).then(...)`-Konfiguration. Siehe die `StorageManager`-Klasse unter Abbildung 9.

5.6.2 Beispiel 2

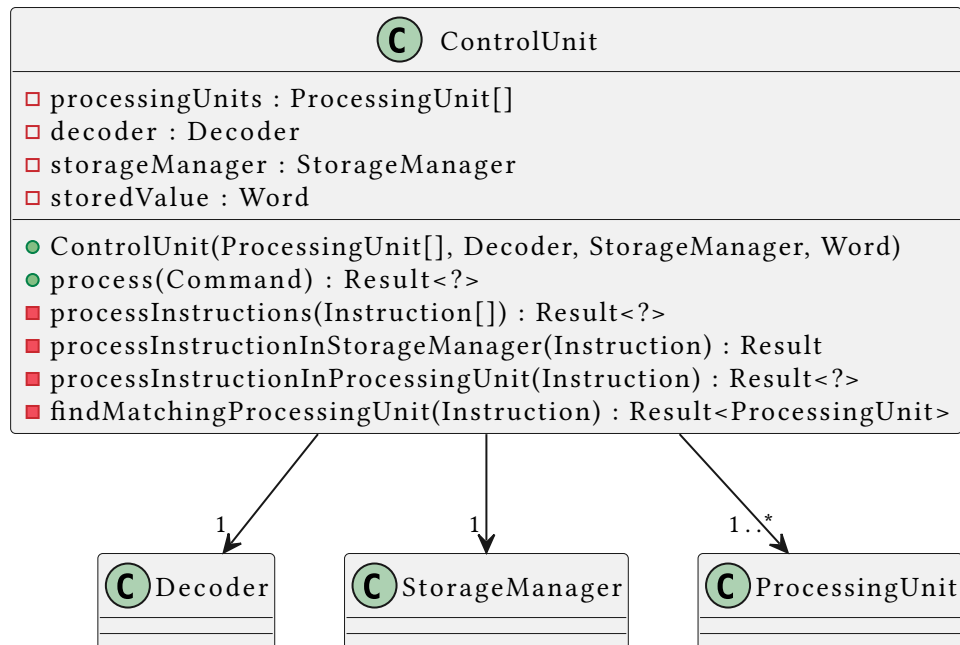


Abbildung 14 – ControlUnit-Klasse UML

In der Testklasse `ControlUnitTest` für die Klasse `ControlUnit`, werden Mocks verwendet. Die Klasse bekommt über ihren Konstruktor mehrere Klassen übergeben (siehe Abbildung 14). Diese wurden für die Tests alle durch Mocks umgesetzt, um gezieltes Testen der Funktionen zu ermöglichen.

6

Kapitel 6: Domain Driven Design

6.1 Ubiquitous Language

Bezeichnung	Bedeutung	Begründung
Keyword	Ist das erste Teil einer Nutzereingabe. Endet ab erstem Leerzeichen.	Identifiziert die Anweisung an das System.
Command	Strukturierte Nutzereingabe. Unterteilt in Keyword und Argumente.	Ist grundlegende Repräsentation der Nutzereingabe.
Instruction	Ein oder mehrere Instructions entstehen aus einem Command. Diese werden genutzt als elementare Operationen auf der CPU	Ebenfalls Teil der Repräsentation der Nutzereingabe.
InstructionValue	Gehört zu einer Instruction. Enthält eine Address oder einen Word eines Commands	Zugehörig zur Trennung der Verantwortlichkeiten innerhalb einer Instruction
Word	Ist die Klasse die den niedrigsten Wert in der CPU symbolisiert. Enthält eine Liste an Wahr- und Falschwerten (Boolean), diese repräsentieren Bits. Ein Word ist ein Byte in dieser Konfiguration	Minimale Darstellung des Speichers.

Bezeichnung	Bedeutung	Begründung
Address	Jeder Speicherplatz in der CPU hat eine adressierbare Address. Sie ist definiert über ein Prefix (bspw. „r“ für Register) und einen Index (bspw. „5“ für die 6. Stelle innerhalb des Registers)	Benötigt zur Adressierung von Speicherplatz.
StoredValue	Ist ein besonderer Speicherplatz innerhalb der ControlUnit, welcher dafür genutzt wird um einen Wert zwischenzuspeichern für weitere Instructions.	Wichtig als Zwischenspeicher für komplexere Abläufe innerhalb oder zwischen mehreren Units.
Unit	Ist eine Klasse welche der ControlUnit unterstellt ist. Diese kann schreiben, lesen und Instructions ausführen, sowie zurückgeben ob sie eine Instruction überhaupt ausführen kann.	Einheiten welche je einen Aufgabenbereich übernehmen innerhalb der CPU.
SubUnit	SubUnits sind spezifisch für die ALU. Die ALU nutzt diese um die unterschiedlichen Teilgebiete der Rechenoperationen zu gliedern. Bspw. ArithmeticSubUnit für die Verarbeitung von Additionen und Subtraktionen oder LogicalSubUnit für Gleichheitsoperationen (bspw. „AND“) und ähnliche.	Wird genutzt zur Aufteilung der einzelnen Aufgabenbestandteile einer Unit.
Operator	Operator ist ebenfalls spezifisch für die ALU. Sie repräsentieren die einzelnen Operationen die die ALU ausführen kann, wie Addition und Subtraktion. Sie können zurückgeben ob sie eine Instruction verarbeiten können und sie im positiven Fall dann verarbeiten in einer weiteren Funktion.	Kleinste Aufgabe einer Unit wird durch einen Operator übernommen.
Decoder	Ist zuständig dafür Commands in Instructions umzuwandeln	Decodierung der Eingaben des Nutzers in Instructions, welche die

Bezeichnung	Bedeutung	Begründung
		CPU verstehen und verarbeiten kann. Ebenfalls wichtig zur Validierung der Nutzereingabe.
ALU	Ist eine Unit und ist zuständig für die arithmetischen Operationen der CPU	Recheneinheit der CPU.
Register	Ist eine Unit welche nur Daten lesen und schreiben kann	Primäre Speichereinheit der CPU.
StorageManager	Ist eine Klasse welche von der ControlUnit verwaltet wird. Sie ist zuständig für Instructions welche Daten verschieben (wie bpsw. „set“, „get“, „store“). Sie erhält Zugriff auf die ProcessingUnits und die StoredValue, um Dateninstructions ausführen zu können.	Nicht in einer herkömmlichen CPU enthalten, aber wichtig für die Trennung der Zuständigkeit innerhalb der ControlUnit.

6.2 Entities

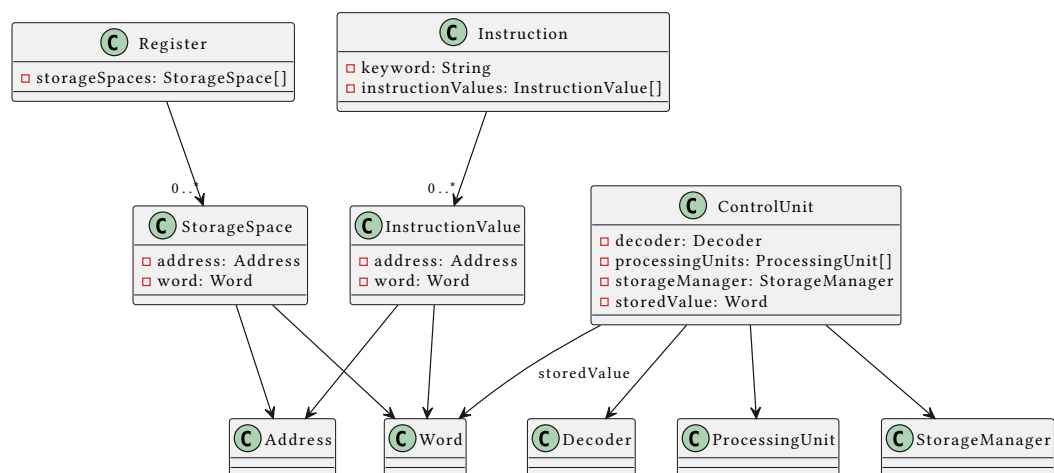


Abbildung 15 – Entities UML

Die in Abbildung 15 gezeigte Klasse `StorageSpace` ist die einzige Klasse, die sich als Entity-Klasse ermitteln lässt. Sie zeichnet sich dadurch aus, dass sie durch eine Adresse eindeutig identifiziert werden kann und der gespeicherte Wert sich ändern kann.

6.3 Value Objects

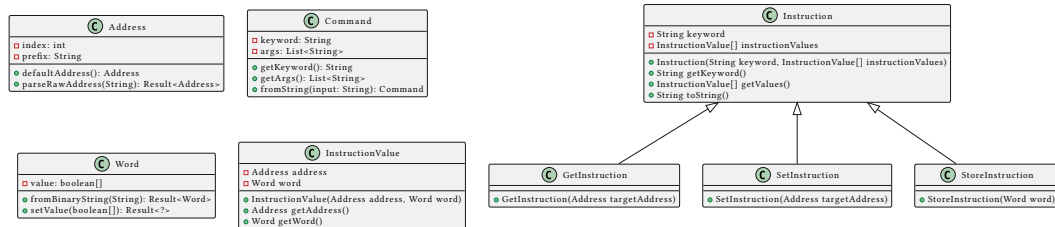


Abbildung 16 – Value Objects UML

Hierbei handelt es sich um Klassen, welche keine eigene Identität haben und nur durch ihre Werte ausgezeichnet werden.

Hierzu gehören folgende Klassen:

- Address
- Command
- Word
- GetInstruction
- SetInstruction
- StoreInstruction
- Instruction
- InstructionValue

6.4 Repositories

Es wurde keine passende Klasse in der Rolle eines Repositories ermittelt.

6.5 Aggregates

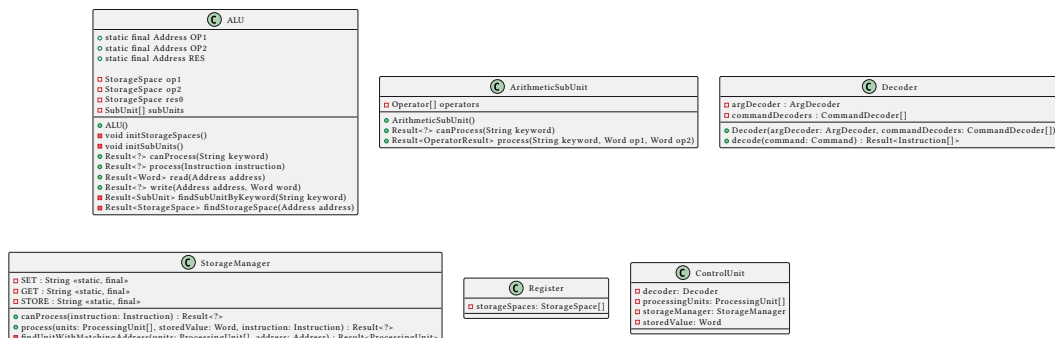


Abbildung 17 – Value Objects UML

Aggregate bringen ähnliche, bzw. zusammengehörige Entities und Value Objects zusammen. Beispiele hierfür in diesem Projekt sind:

- ControlUnit
- Decoder
- StorageManager
- ArithmeticSubUnit
- CPUSimulator
- Register



Kapitel 7: Refactoring

7.1 Code Smells

7.1.1 Code Smell 1

Der `StorageManager` enthält einen Code Smell (siehe GitHub). Dieser hat eine verhältnismäßig große Methode `process`. Diese verarbeitet alle drei Fälle `set`, `get` und `store`. Das bringt folgendes Problem mit sich. Es wird nicht das Single-Responsibility-Principle, sowie das Open-Closed-Principle erfüllt.

Verbessert werden kann das durch das Verteilen der Zuständigkeit auf einzelne Klassen. Somit würde der `StorageManager` eine Liste an Klassen halten, welche sich je um einen Fall kümmern. Diese Liste könnte einfach erweitert oder verkleinert werden.

7.1.2 Code Smell 2

Ein weiteres Problem zeigt sich in der ursprünglichen `OutputHandler`-Klasse. Diese war dafür zuständig, Textausgaben an den Nutzer zu übermitteln, übernahm jedoch zugleich auch die Aufgabe, Ergebnisse (`Result`-Klasse) semantisch zu interpretieren und entsprechend darzustellen. Damit wurde das Single-Responsibility-Principle verletzt, da die Klasse zwei verschiedene Verantwortlichkeiten vereinte: die technische Ausgabe sowie die inhaltliche Aufbereitung von Ergebnissen.

Die Vermischung dieser Verantwortlichkeiten führte nicht nur zu reduzierter Lesbarkeit, sondern erschwerte auch die Testbarkeit und Wiederverwendbarkeit der Komponente. Eine saubere Trennung dieser Aufgaben wird notwendig, um die Wartbarkeit und Erweiterbarkeit des Systems zu gewährleisten. Um dieses Problem zu lösen kann eine weitere Klasse entworfen werden um separat dieses Problem der Verarbeitung der Result-Klasse zu behandeln.

7.2 2 Refactorings

7.2.1 Refactoring 1

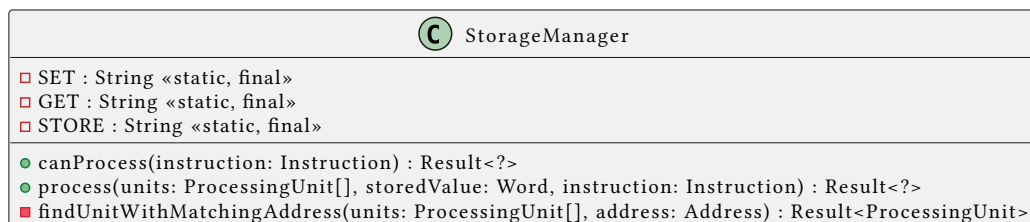


Abbildung 18 — StorageManager vor Refactoring 1

Abbildung 18 zeigt wie die StorageManager-Klasse vor dem Refactoring 1 operiert hat. Die vollständige Verantwortlichkeit über alle drei Funktionen set, get und store lagen innerhalb der Klasse. Durch das Refactoring 1 wurde dieses Designkonzept geändert, aber die selbe Funktionalität beibehalten.

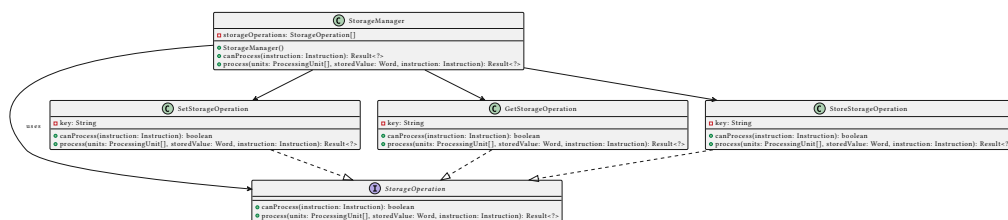


Abbildung 19 — StorageManager nach Refactoring 1

Abbildung 19 zeigt welche Auswirkungen das Refactoring 1 hatte auf die Struktur von der StorageManager-Klasse.

[Link zum Commit](#)

7.2.2 Refactoring 2

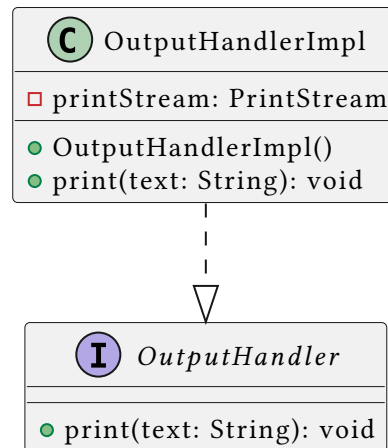


Abbildung 20 – OutputHandler vor Refactoring 2

Abbildung 20 zeigt den Stand vor dem Refactoring 2. Der Grund für das Refactoring liegt darin, dass der `OutputHandler` zuvor zwei Aufgaben übernommen hat. Hierbei handelte es sich um das Verarbeiten der `Result`-Klasse und die Ausgabe selbst. Diese Aufgaben sollten isoliert voneinander verarbeitet werden. Hierfür wurde dann das Refactoring 2 durchgeführt.

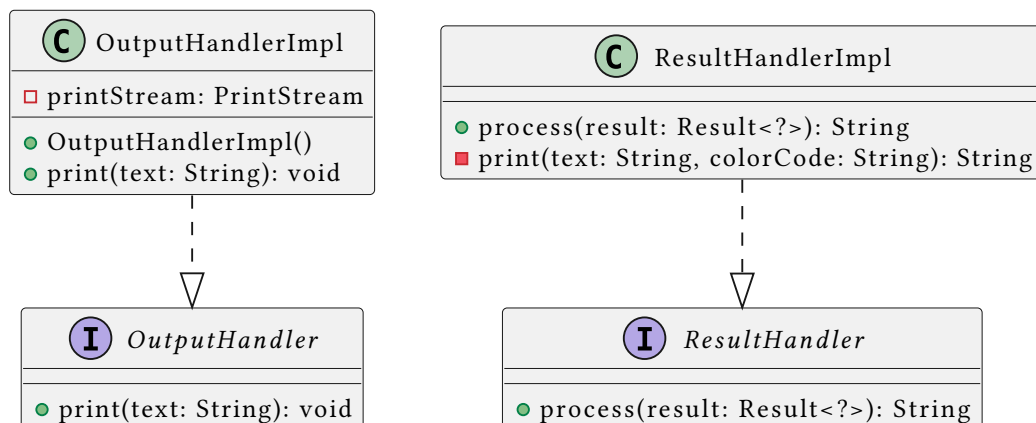


Abbildung 21 – OutputHandler nach Refactoring 2

Abbildung 21 zeigt, dass eine weitere Klasse `ResultHandler` entworfen wurde, um die Aufgabe des Verarbeitens der `Result`-Klasse zu übernehmen. Somit wurde die Logik getrennt.

[Link zum Commit](#)

8

Kapitel 8: Entwurfsmuster

8.1 Entwurfsmuster: Builder

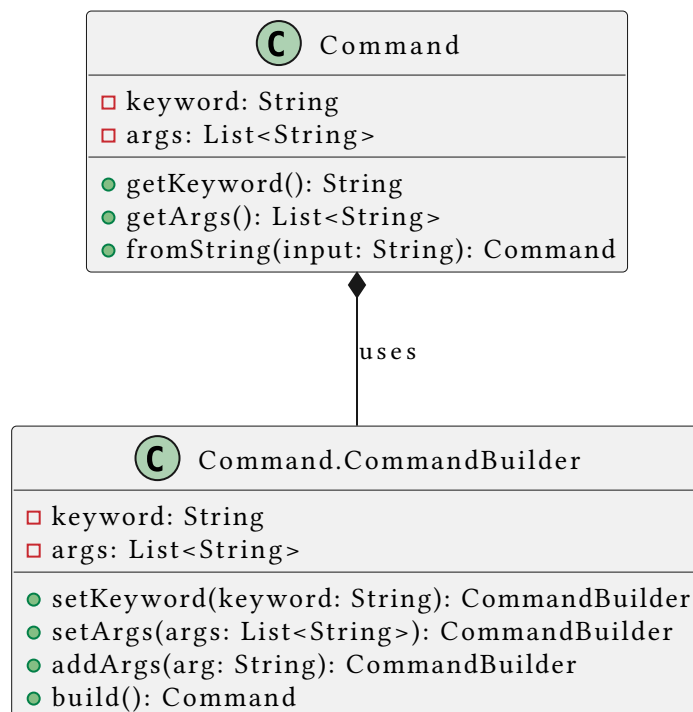


Abbildung 22 — Command UML

Ein Command (siehe Abbildung 22) kann über einen Erbauer, den Commandbuilder, gebaut werden. Er ermöglicht das Commands sequenziell erstellt werden können.

8.2 Entwurfsmuster: Strategie

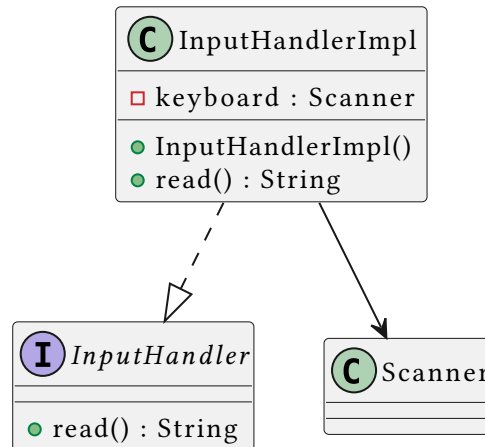


Abbildung 23 – InputHandler UML

Der `InputHandler` ist ein gutes Beispiel für das Entwurfsmuster der Strategie. Bei `InputHandler` handelt es sich um ein Interface, welches voraussetzt, dass ein `String` bei Aufruf der `read`-Methode zurückgeliefert wird. Somit wird ermöglicht, dass unterschiedliche Eingabequellen einfach realisiert und eingebunden werden können.

A Literatur

B Abkürzungen

API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer

C Glossar

Komponente	Ein Architekturbaustein. Zusammengesetzte Komponenten bestehen aus weiteren Subkomponenten. Einfache Komponenten sind nicht weiter unterteilt.
Softwareschnittstelle	Ein logischer Berührungspunkt in einem Softwaresystem: Sie ermöglicht und regelt den Austausch von Kommandos und Daten zwischen verschiedenen Prozessen und Komponenten.

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

CPU Simulation

enthält unternehmensinterne bzw. vertrauliche Informationen der Atruvia AG, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Informatik der Duale Hochschule Baden-Württemberg Karlsruhe vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (Atruvia AG) vorliegt.

Karlsruhe 15.06.2025

Marc Schillinger

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

CPU Simulation

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Karlsruhe 15.06.2025

Marc Schillinger